Phase 0: Setup & How a Web App Works

What we built

- `app/` : a React + TypeScript project created with **Vite**
- **Tailwind CSS** for styling
- `app/src/App.tsx` : a dark “shell” (a header and 3 empty panels: Watchlist, Chart, Order Panel)

Run it:

```
cd app
npm run dev      # starts the dev server → open http://localhost:5173
```

Concepts (theory only)

HTML / CSS / JavaScript

Layer	Job	Analogy
HTML	**Structure**: what is on the page (headings, buttons, divs)	Skeleton
CSS	**Looks**: colours, spacing, layout	Skin & clothes
JS	**Behaviour**: reacting to clicks and data	Muscles & brain

The browser turns HTML into a tree called the **DOM** (Document Object Model). JS changes the page by changing the DOM.

Node.js & npm

- **Node.js** runs JavaScript outside the browser. We use it only for *tooling* (dev server, build, tests). Our app itself still runs in the browser.
- **npm** is the package manager, like `pip` for Python or Maven for Java.
  - `package.json` lists the dependencies and the scripts (`dev` , `build` , `test`)
  - `node_modules/` holds the downloaded packages (never commit it)
  - `dependencies` ship to users (react). `devDependencies` are build-time only (vite, typescript)

Vite (the build tool)

- **Dev server:** serves your code instantly and does **HMR** (Hot Module Replacement): save a file and the browser updates without a full reload.
- **Build:** `npm run build` bundles and minifies everything into `dist/` , which is static files you can host anywhere.

- Why Vite and not Create React App: much faster, because it uses native ES modules in dev. CRA is deprecated.

TypeScript

- JavaScript **plus types**. You catch bugs at compile time, not in front of users.
- `tsc` checks the types, then the types are *erased*. The browser only ever runs plain JS.
- Key ideas: `type` / `interface` (the shape of an object), union types (`'MARKET' | 'LIMIT'`), generics (`Array<number>`).
- Files: `.ts` = plain TS, `.tsx` = TS with JSX (React markup).

Tailwind CSS

- **Utility-first:** instead of writing CSS classes yourself, you combine tiny ready-made classes in the HTML: `className="rounded-lg border bg-panel p-3"` means rounded corners + border + background + padding.
- **Design tokens:** in `src/index.css` the `@theme` block defines our colours once (`--color-up` , `--color-down` …). That gives us classes like `text-up` / `bg-panel` . Change a token and the whole app changes.
- Responsive prefixes: `lg:grid-cols-[...]` means “apply only on large screens”. (More in Phase 8.)

How the app boots

```
index.html  ── loads ──>  src/main.tsx  ── renders ──>  <App />  into <div id="root">
```

This is a **Single Page Application (SPA)**. There is one HTML page, and React draws and redraws everything inside `#root` .

Project structure (so far)

```
trading-dashboard/
├── PLAN.md            ← the 20h roadmap
├── notes/             ← your study notes (this folder)
└── app/
    ├── index.html     ← the single HTML page
    ├── package.json   ← deps + scripts
    ├── vite.config.ts ← Vite + React + Tailwind plugins
    └── src/
        ├── main.tsx   ← entry point
        ├── App.tsx    ← root component (the shell)
        └── index.css  ← Tailwind + colour tokens
```

Where the data comes from

Part	Source
Watchlist prices, candles, order book	**Real**, from Binance's public REST + WebSocket APIs (no key needed)
Placing orders	**Simulated**: filled at the real live price (real orders need API keys + money)
Positions & P&L	Calculated from simulated orders, using **real** live prices
Option chain	Strikes built around the real spot price, Greeks from Black–Scholes; IV & OI are modelled
Phase 1 watchlist	Hard-coded mock data on purpose (learn React first, live data comes in Phase 2–3)

> **Interview line:** "Market data is live from Binance's public WebSocket and REST APIs. Order execution is simulated client-side at the live price, since real trading needs authenticated keys."

Resume mapping

- "**TypeScript**", "**Vite**", "**Tailwind CSS**", "**HTML5, CSS3**": the setup is done, and you can now explain what each one does.

Interview Qs

1. *What does Vite do?* It's a dev server with HMR plus a production bundler. It's fast because dev serves native ES modules without bundling.
2. *Why TypeScript?* Static types catch bugs early, give better autocomplete, and make refactoring safer. They're erased at build time.
3. *Why Tailwind over plain CSS?* No naming of classes, no dead CSS, consistent design tokens, and styles sit next to the markup.
4. *What is an SPA?* One HTML page. JS renders and updates views on the client without a full page reload.